Data Mining Project
Business Understanding, Data Understanding, Data Preparation, and Modeling

Bulat Sharipov — DS-01

May 2026

1 Business Understanding

1.1 Determine Business Objectives

Modern code assistants and automated review systems require data that describes how software changes are actually produced. Public code datasets usually contain source files, commits, or code–text pairs, but they often miss the surrounding workflow: the pull request, review discussion, requested changes, and final diff. This project addresses that gap by constructing a curated dataset of public GitHub pull request workflows.

Business objective: create a reusable dataset product for machine learning teams that build code review automation, coding assistants, and developer-agent systems. Each record should represent one pull request workflow and combine pull request metadata, review activity, discussion, changed files, code diff, quality metadata, and provenance.

Business success criteria: the project is successful if it produces a multi-repository dataset with at least 5,000 high-quality workflows, removes obvious noise such as bots and generated-only changes, supports a realistic downstream prediction task, and demonstrates that the dataset contains learnable signal beyond simple baselines.

1.2 Assess Situation

The main data source is GH Archive, which provides public GitHub event history and makes it possible to discover merged pull request candidates at scale. The GitHub REST API is then used to enrich each candidate with complete pull request metadata, changed files, reviews, discussion comments, and unified diffs. The work is constrained to public repositories and local compute, so the collection strategy must be efficient and reproducible.

The main risk is data noise. Public GitHub activity contains bots, trivial edits, generated files, documentation-only changes, and workflows with too

little review content. This risk is handled by filtering, quality metadata, and repository-level splitting. Another important risk is label imbalance: the later modeling target, `review_concern`, is positive for most records. Therefore, accuracy alone is not a meaningful success measure; macro-F1, balanced accuracy, and ROC-AUC are more informative.

The expected value of the dataset is saved engineering effort. A team that wants workflow-level GitHub data would otherwise need to implement event discovery, API enrichment, filtering, schema design, and quality checks. A compact but curated dataset of roughly 10,000 enriched workflows can therefore be positioned as a niche dataset product for AI engineering teams.

1.3 Determine Data Mining Goals

The first data mining goal is corpus construction. The project should produce at least 5,000 accepted pull request workflows, with a target close to 10,000 raw enriched records. A workflow is accepted when it represents a merged pull request with source-code changes, non-empty source patches, review and discussion activity, and no strong bot, documentation-only, generated-only, or trivial-change pattern.

The second data mining goal is supervised prediction of `review_concern`. The model receives only pull-request-intrinsic information available at opening time, including metadata, changed-file summaries, and capped code-diff embeddings. It predicts whether later review activity will contain a substantive concern signal, defined by a `CHANGES_REQUESTED` review or at least two meaningful review comments.

Data mining success criteria: the prepared dataset should pass the size and quality thresholds, and the best held-out model should beat a majority-class baseline on macro-F1 and random ranking on ROC-AUC. These criteria are intentionally modest because the modeling stage is used to validate dataset usefulness, not to claim production readiness.

1.4 Produce Project Plan

The project follows the CRISP-DM sequence. First, the business objective and downstream task are fixed. Second, candidate pull requests are collected and enriched. Third, the data is explored for scale, completeness, size, review activity, language diversity, and quality limitations. Fourth, the raw records are converted into a modeling table with leakage-safe features and repository-level splits. Fifth, several baseline and non-linear models are trained and compared. Finally, the results are evaluated against the technical success criteria and used to decide whether another iteration is required.

The selected tools are standard and local: Python for data processing, GH Archive and the GitHub REST API for data acquisition, Pandas and NumPy for data preparation, ModernBERT embeddings for compact diff representation, scikit-learn for baselines, and XGBoost for the main non-linear model.

2 Data Understanding

2.1 Collect Initial Data

The raw dataset contains enriched public GitHub pull requests. Candidate pull requests were discovered from GH Archive by selecting merged pull request events and joining them with review and discussion activity for the same repository and pull request number. The candidate filter kept medium-sized pull requests: 2–30 changed files, 50–2000 changed lines, at least two discussion comments, at least one formal review, and at least two review comments.

Each candidate was then enriched through the GitHub REST API. The enrichment added the pull request object, changed-file list, formal reviews, inline review comments, pull request discussion comments, full unified diff, API error metadata, and retrieval timestamp. This produces a rich raw layer, but not yet the final curated modeling dataset.

2.2 Describe Data

The collected raw file contains 10,000 enriched pull request records and occupies about 1.78 GB. It covers 6,033 repositories and 8,610 authors. Most pull requests were created and merged from March to May 2025, while enrichment was retrieved on May 1–2, 2026.

The dataset is broad but not uniform. Large active repositories naturally contribute more pull requests, so the collection has a long-tailed repository distribution. The largest repositories include `llvm/llvm-project`, `WebKit/WebKit`, and `Automattic/wp-calypso`. This concentration is expected for public GitHub data and motivates the repository-level split used later.

2.3 Explore Data

The first exploration question is whether the collection has enough scale and temporal coverage. The timeline plot shows that the selected pull requests are concentrated in a recent 2025 window, which is acceptable for the current project because the goal is a contemporary workflow dataset rather than a historical trend analysis.

The second question is whether the enriched fields are sufficiently complete. Pull request objects are present in 100.00% of records, file lists in 99.99%, full diffs in 99.96%, formal reviews in 99.91%, review comments in 99.82%, and pull request comments in 99.78%. Source files appear in 83.61% of records. This is enough for analysis, although records without source files or source patches must be marked as lower quality.

The pull requests are mostly small to medium-sized, as intended by the candidate filter. The median pull request has 7 changed files and 261 changed lines. The distributions remain skewed, especially for additions, deletions, commits, and full-diff character count; therefore, the preparation stage uses capped tokenized diff embeddings instead of feeding complete diffs directly into a model.

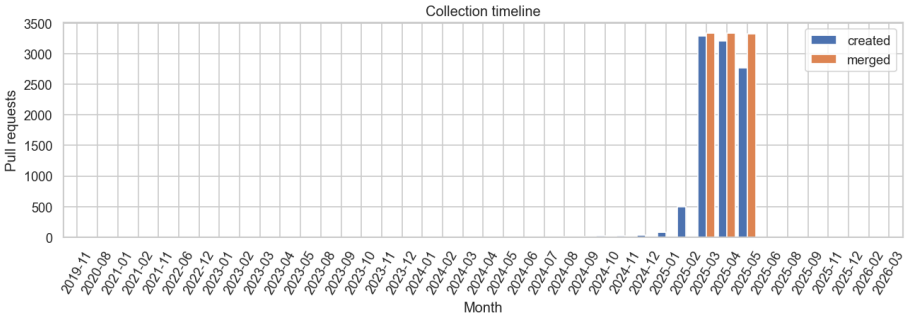

Figure 1: Pull request creation and merge month distribution.

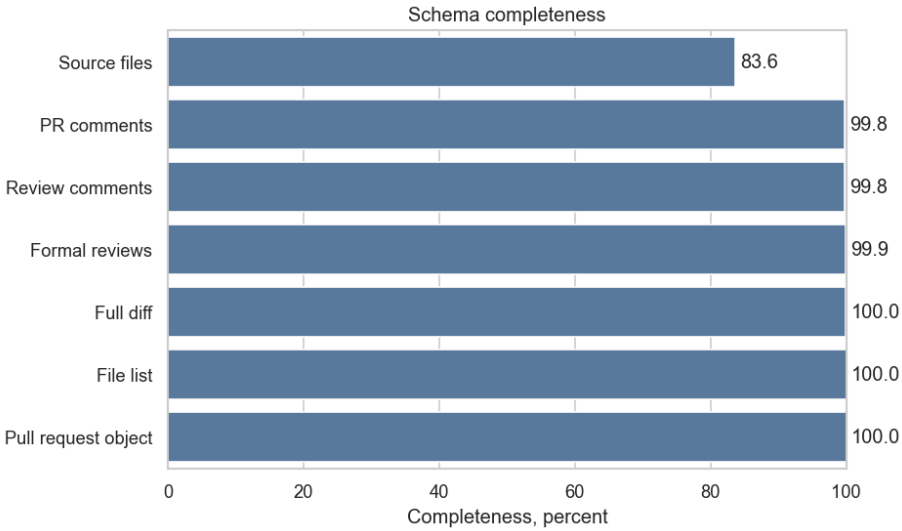

Figure 2: Completeness of key enriched fields.

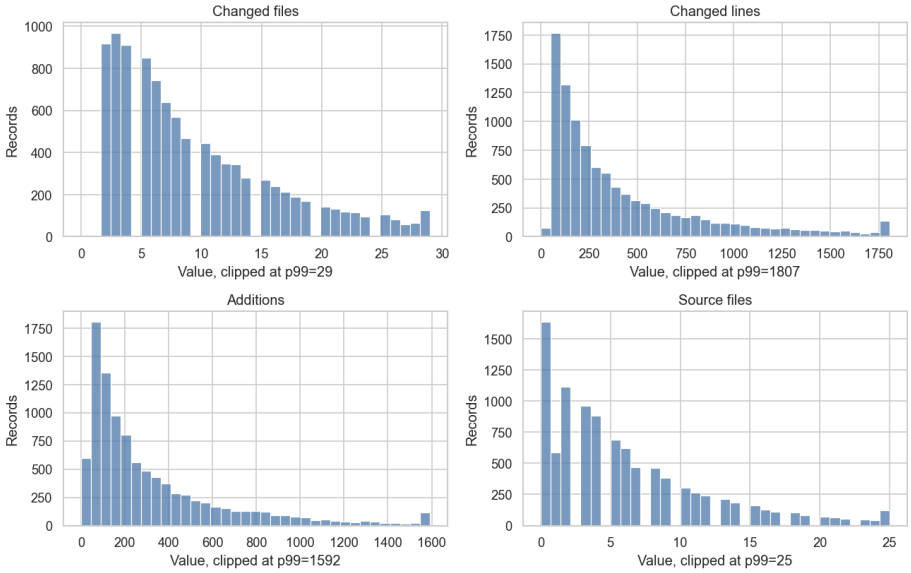

Figure 3: Pull request size distributions.

Review activity is also skewed but sufficient. The median record has 6 formal reviews, 6 inline review comments, 6 meaningful review comments, and 3 pull request discussion comments. This supports the downstream target because most records contain enough reviewer activity to decide whether a concern was raised.

The language distribution is diverse. TypeScript, Python, Java, Go, Rust, JavaScript, C++, C/C++, C#, and Kotlin are the most common source languages. This is useful for the dataset goal because the model should not be tied to one ecosystem.

Under the current quality filters, 7,373 records pass and 2,627 are rejected. The dominant rejection reasons are missing source-code patches, missing source files, insufficient discussion, insufficient meaningful review comments, and documentation-only changes. These failures are expected in public repository data and justify retaining quality metadata instead of assuming that every enriched pull request is modeling-ready.

2.4 Verify Data Quality

The raw enrichment quality is sufficient for the next CRISP-DM stage. The dataset has enough scale, broad repository coverage, diverse languages, and high completeness for core pull request, review, comment, and diff fields. The main limitations are not missing metadata but unsuitable records: non-source changes, absent source patches, short discussions, and weak review-comment content.

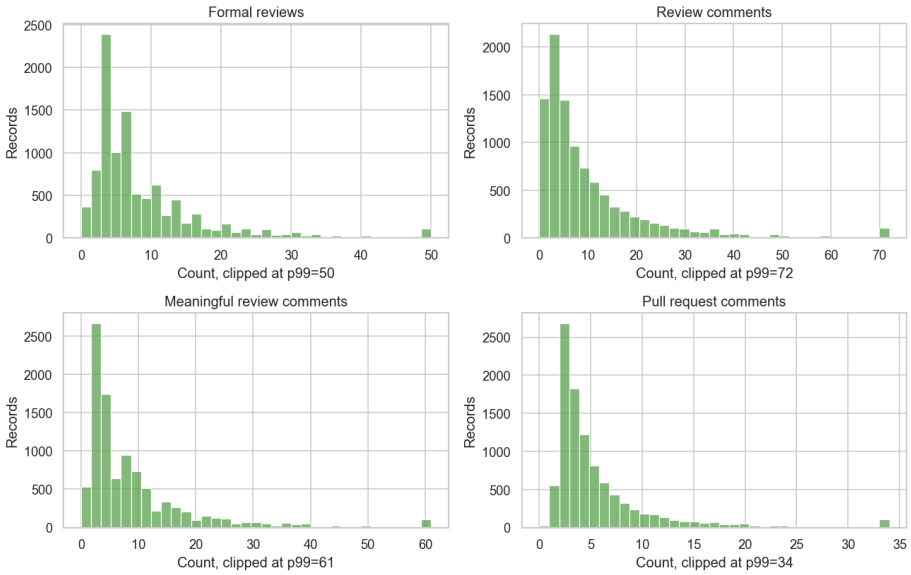

Figure 4: Review and discussion activity distributions.

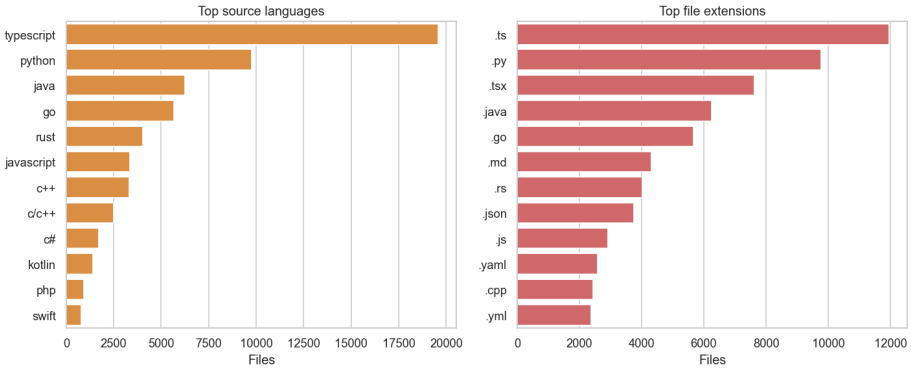

Figure 5: Top source languages and file extensions.

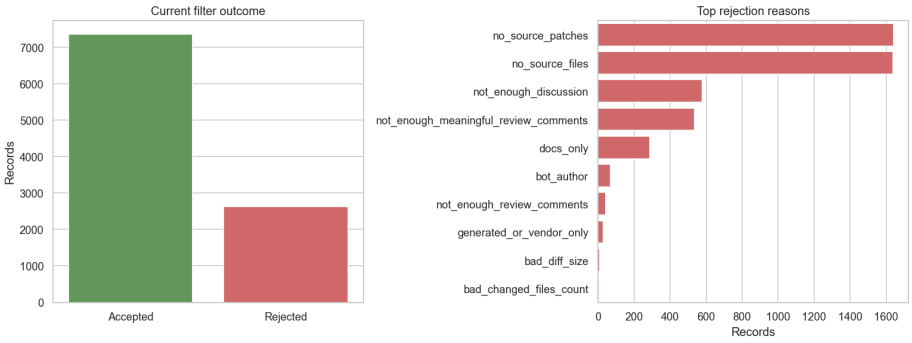

Figure 6: Accepted versus rejected records and top rejection reasons under current filters.

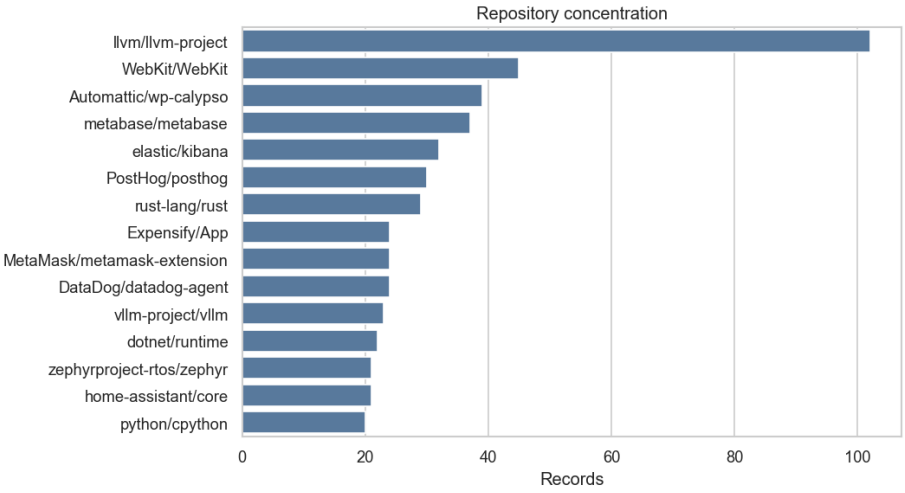

Figure 7: Top repositories by raw record count.

Therefore, the raw file should not be used directly for modeling. It should first be transformed into a leakage-safe table, with quality flags retained for audit and with review-derived variables excluded from the feature matrix. The data is suitable for preparation and baseline modeling, but the later model should be interpreted as a proof of signal rather than a production review-concern detector.

3 Data Preparation

3.1 Select Data

The preparation stage uses all 10,000 enriched pull request records. The modeling unit remains one pull request workflow. The target is `review_concern`: a row is positive if later review activity contains a `CHANGES_REQUESTED` review or at least two meaningful review comments, negative if review activity exists without those signals, and excluded if review activity is absent. Only 9 rows lack review activity, leaving 9,991 labeled rows.

The accepted/rejected quality flag is not used as the prediction target. It is retained only as audit metadata. This distinction is important because the project goal is not to predict whether a row passes curation, but whether a pull request will later receive substantive review concern.

3.2 Clean Data

The raw rows are not silently discarded. Instead, quality metadata is attached to each record. The main quality problems are bot authors, documentation-only changes, generated or vendor-only changes, missing source files, missing source patches, insufficient discussion, and insufficient meaningful review comments. These fields help audit the dataset but are not used as modeling features.

Review-derived counts and review states are also excluded from the feature matrix. They are useful for defining the label, but they occur after the pull request is opened. Including them as features would leak the answer into the model and overstate performance.

3.3 Construct Data

Each row is converted into three groups of features. The first group contains pull-request-intrinsic numeric features: changed files, additions, deletions, diff size, commit count, title and body length, source-file counts, source-patch counts, and capped source-patch token counts. The second group contains one-hot indicators for top source language and author association. The third group is a 768-dimensional ModernBERT embedding of the source-code diff.

The embedded text is constructed from source-code `files[].patch` values. Because full diffs can be large, each row is capped at 1,000 ModernBERT tokens before embedding. This reduces total embedded content from 51.4 million raw tokens to 8.0 million capped tokens. The cap is a practical compromise: it keeps

local embedding feasible while preserving a compact semantic representation of the changed code.

3.4 Integrate Data

The prepared table integrates pull request metadata, changed-file summaries, source patches, constructed labels, quality metadata, and diff embeddings into one row per pull request. The split is performed at repository level with a fixed seed. This prevents pull requests from the same repository from appearing in both training and evaluation splits, which would otherwise inflate performance because repositories share conventions, file structure, and review style.

The resulting labeled split contains 8,056 training rows, 954 validation rows, and 981 test rows. The class imbalance is strong: the training set contains 367 negative and 7,689 positive examples. This imbalance is carried into modeling and is the main reason why macro-F1 and balanced accuracy are reported alongside ranking metrics.

3.5 Format Data

The final modeling matrix contains 802 numeric features: 14 pull-request numeric features, 20 one-hot categorical indicators, and 768 embedding dimensions. The same feature schema is used for training, validation, and test data, so the matrices can be passed directly to standard `.fit()` and `.predict()` workflows.

Overall, data preparation converts the raw enriched pull request collection into a leakage-safe modeling dataset. The main preparation decisions are to retain all raw rows for audit, exclude post-review variables from the feature matrix, cap long diffs to 1,000 tokens, represent code changes with ModernBERT embeddings, and split by repository to obtain a more realistic estimate of generalization.

4 Modeling

4.1 Select Modeling Technique

The modeling task is binary classification for `review_concern`. A positive example is a pull request that later receives a substantive concern signal during review; a negative example has review activity but no such signal. The task is difficult because the model only sees information available at pull request opening time, while the label is determined by later reviewer behavior.

Five model candidates were compared. The majority-class baseline measures how much can be achieved by exploiting class imbalance only. Logistic regression is used as a linear reference, first on tabular pull request features and then on the full feature set. XGBoost is used as the main non-linear candidate because it can combine numeric metadata, sparse indicators, and dense diff embeddings.

The all-feature models use 802 features, including the 768-dimensional capped ModernBERT representation of the source-code diff.

4.2 Generate Test Design

The repository-level split from data preparation is preserved: 8,056 training rows, 954 validation rows, and 981 test rows. This design is important because pull requests from the same repository share code style, review norms, file organization, and ownership patterns. A random row-level split would make the task easier and would likely overestimate performance.

Model selection is based on validation ROC-AUC because the most defensible first use of the model is ranking pull requests by concern likelihood. Thresholded metrics are still reported because a practical classifier must eventually make class decisions. The threshold is chosen on the validation set to maximize macro-F1, which gives the rare negative class more influence than accuracy would.

4.3 Build Model

The validation results show that the capped diff embedding adds useful signal. Logistic regression improves when embeddings are added, and XGBoost with all features performs best by ROC-AUC. However, the improvement over simpler models is moderate, which already suggests that the representation captures only part of the review-concern signal.

Model	ROC-AUC	PR-AUC	Macro-F1	Balanced Acc.
Majority baseline	0.500	0.959	0.490	0.500
Logistic, tabular	0.539	0.965	0.511	0.517
Logistic, all features	0.599	0.973	0.517	0.522
XGBoost, tabular	0.567	0.969	0.506	0.508
XGBoost, all features	0.638	0.976	0.532	0.532

Table 1: Validation results for `review_concern`.

4.4 Assess Model

The selected model is XGBoost with tabular features and code-diff embeddings. On the held-out test split it reaches ROC-AUC 0.628, PR-AUC 0.976, macro-F1 0.537, and balanced accuracy 0.534. The test confusion matrix is $[[4, 36], [30, 911]]$.

The high PR-AUC should be interpreted carefully. The positive class dominates the dataset, so even the majority baseline has PR-AUC 0.959. ROC-AUC, macro-F1, and balanced accuracy are more informative here. They show that the model has learned some ranking signal, but the separation between concerned and non-concerned pull requests remains weak.

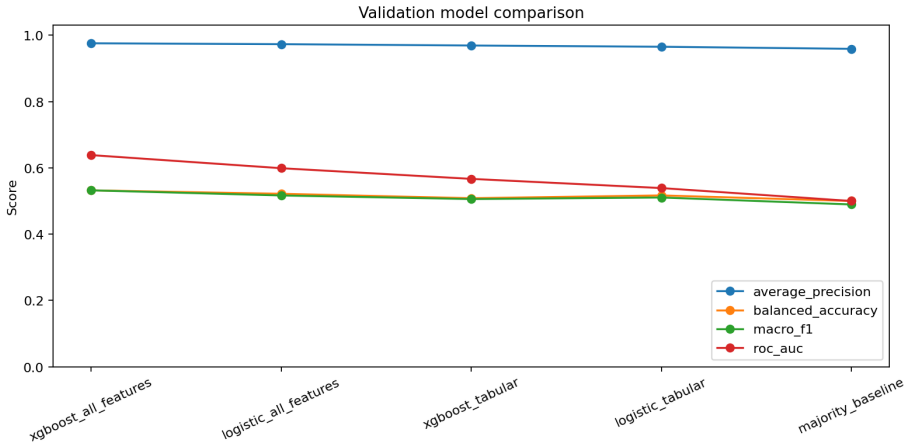

Figure 8: Validation metric comparison across model candidates.

The confusion matrix makes the limitation clear. The model correctly identifies most positive examples, but it detects only 4 of 40 negative examples in the test split. In other words, the model is much better at preserving recall for likely review concerns than at confidently recognizing pull requests without concern signals. This is acceptable as a baseline proving that the dataset contains signal, but it is not reliable enough for an automated decision system.

The likely reason is that review concern is not determined only by local diff text. Reviewers react to hidden project conventions, architectural fit, missing tests, API compatibility, maintainability, prior knowledge of the codebase, and the intent behind the change. A 1,000-token pooled diff embedding compresses much of this context and may miss long-range dependencies across files. Stronger performance would likely require a more powerful language-model setup: longer-context code-aware encoders, fine-tuning on review outcomes, structured discussion context, repository history, or an LLM-style model that can reason over the full pull request trace rather than only a capped diff representation.

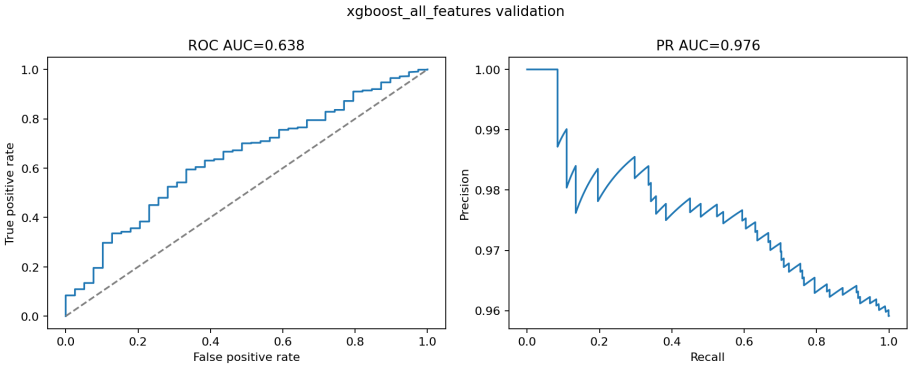

Figure 9: Selected model validation ROC and precision–recall curves.

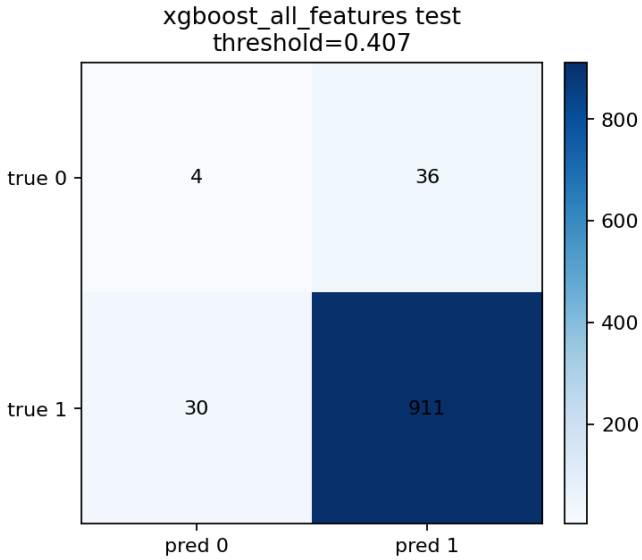

Figure 10: Selected model test confusion matrix.